AutoFish Project — Complete Guide (Layman, from Scratch)

What this document is. A complete, plain-language explanation of the whole project, written so you can walk into the meeting and explain or defend any part of it. It also covers the new directions the supervisor raised, and a full beginner's guide to *hyperparameters* and how we would tune them for each model.

Honesty labels used throughout: - ✅ **FACT** — verified from our code, configs, or result files. - 📊 **RESULT** — a finding that follows from our numbers. - 💡 **HYPOTHESIS** — a possible explanation we did **not** prove; say it as a maybe. - ⚠️ **NOT DONE / LIMITATION** — not yet done or not yet validated.

Authors: Abu Bakar, Laksh Jiwani, Shahman Butt · Supervisor: Bohan Zhuang, M.Sc. · Professor: Stefan Oehmcke · University of Rostock (VACOT). Repo: https://github.com/Shahman-Butt/AreaSeminarFishLength

---

PART A — The project in one breath

We taught a computer to measure a fish's **length in centimetres** from a photo. We first **reproduced** the published AutoFish result (to prove our setup is correct), then we **swapped the "eye" of the system** for several modern AI models to see if any could measure fish better. Result: the original small network (MobileNetV2) stayed best; the big "foundation models" did not beat it.

PART B — The building blocks (plain language)

The task. Input: a cropped photo of one fish plus its bounding box (the rectangle around it). Output: one number, the length in cm. Because the answer is a continuous number, this is **regression** (not classification, which would pick a category, and not detection or segmentation, which find *where* the fish is). ✅

The machine has three parts, and we only ever change the first: 1. **Encoder ("the eye")** — turns the fish image into a list of numbers (features). We swap this. 2. **Bounding-box hint** — 4 numbers giving the fish's real position and size, because resizing the crop hides its true scale. ✅ (`bbox_input: true` in every config.) 3. **Regression head ("the decision-maker")** — a small network turning features + hint into the final length number. ✅

Training words, in plain terms: - **Epoch** = one full pass through all 10,759 training fish. Baseline ran 200; the swaps ran 100. ✅ - **Batch** = how many fish the model looks at before adjusting once. Baseline 32; swaps 16. ✅ - **Learning rate** = the size of each adjustment step. `1e-3` = 0.001 (normal), `1e-4` = 0.0001 (gentle), `1e-6` = 0.000001 (tiny). ✅ - **Checkpoint** = a saved copy of the model; we keep the one that scored best on the validation set. ✅ - **Frozen / partial / full** = how much of the eye we allow to change: nothing, just the last layer, or everything. ✅

PART C — The data, and the numbers that confuse people

The studio: 454 real fish, handled in 25 groups, photographed 1,500 times → 18,157 "fish-in-a-photo" records (annotations), each with a hand-measured true length. ✅

Easy vs hard photos: Set1 and Set2 = fish laid out separately (**non-occluded**, easy); "All" = fish overlapping (**occluded**, hard). ✅

The split (by whole groups, never by photo): ✅

Bucket	Job	Groups	Fish-images	Unique fish
Train	model learns	15	10,759	269
Validation	pick best version	5	3,639	91
Test	final exam (once)	5	3,759	94

Why group split: the same fish appears in many photos; a random photo split would put the same fish in train and test, letting the model "recognise" it instead of measuring → **data leakage**. ✅

The 1,879 / 1,880 numbers are TEST numbers, the easy/hard halves of the 3,759 test set:

Test condition	Count	Paper	Ours (MobileNetV2)
Non-occluded (easy)	1,879	0.62 cm	**0.633 cm**
Occluded (hard)	1,880	1.38 cm	0.909 cm
Full test (all)	3,759	(not reported)	**0.771 cm**

The poster's headline **0.771 cm** is our own **full-test** number (all 3,759 fish), used to rank all models. It sits between the easy (0.633) and hard (0.909) scores. ✅

Fish 113 (the leak we removed): one fish had 40 images in a test group and 1 in a training group. We removed that 1 stray image (annotation 3759, logged in `exclusions.json`); that is why non-occluded test = 1,879, not 1,880. Zero leaks remain. ✅

PART D — The models, and what won

Model	Type	Pretraining	Best full-test MAE
MobileNetV2	CNN (small)	supervised ImageNet	**0.771** 🥇
ConvNeXt-Tiny	CNN (modern)	supervised ImageNet	0.914 🥈
CLIP ViT-B/32	Transformer	image–text pairs	0.958 / 1.002
DINOv2 ViT-S/14	Transformer	self-supervised, no labels	1.439 (best variant)

📊 The two **supervised CNNs won**, CLIP came next, DINOv2 last. Partial (last-block) fine-tuning improved both foundation models. No foundation model beat the baseline.

💡 *Hypotheses for why* (say as hypotheses): DINOv2's single summary vector favours "what is this" over precise size; CLIP's huge image-text training may keep more size/shape cues; supervised CNNs are simply well-matched to this precise task. None of these is proven.

PART E — NEW: the deeper analysis the supervisor asked for

We built this from our saved per-fish predictions — **no retraining needed** (`scripts/error_analysis.py`, figures in `results/qualitative/`).

When does each model win?

📊 **CLIP actually beats MobileNetV2 on 38.6% of individual fish**, even though MobileNetV2 wins on average — and CLIP wins **more often on occluded fish (40.5%)** than non-occluded (36.7%). So the "MobileNetV2 is best" story is true *on average* but not for every fish: on hard, overlapping cases CLIP is competitive. ✅ 📊

Which fish are hardest (for everyone)?

📊 By length, the **smallest and largest fish** are hardest (a U-shape): e.g. MobileNetV2 errs 0.94 cm on the shortest fish and 1.09 cm on the longest, but only ~0.6 cm in the middle. By species, **hake** and the **"other"** category are hardest for all models.

The clearest failure: DINOv2 on small flatfish

📊 DINOv2's biggest mistakes are all **small wide-bodied flatfish** (22–25 cm) that it over-predicts to 34–38 cm, while MobileNetV2 gets them right. 💡 Likely it confuses the fish's **width/area** for length on flat species. This is shown in `results/qualitative/mobilenet_vs_dino.png`.

Visual figures produced (for the poster)

1. `dataset_examples.png` — example masked crops (easy + hard) so viewers see the task.
2. `mobilenet_vs_dino.png` — cases where MobileNetV2 is right and DINOv2 fails badly.
3. `clip_wins.png` — occluded fish where CLIP beats MobileNetV2.

PART F — NEW: the supervisor's questions, answered

Q: Did you also test species/class identification and mask segmentation? Same trend? ⚠️ **Not yet.** We only did **length regression**. The dataset *has* species labels and masks (we *used* the masks to crop), but we did not train a species classifier or a segmentation model. So we cannot yet say whether the "CNN beats foundation models" trend holds there. **This is exactly the new work we are starting** (species classification first, segmentation is a larger task). Honest answer: "*Only length so far; species classification is running/next, segmentation is scoped as a bigger task.*"

Q: Did you try DINOv3 as a backbone? ⚠️ **No** — we used **DINOv2 ViT-S/14**. DINOv3 is a good next test and is on our list. ✅

Q: Did you use only the CLS token for DINOv2, or also patch tokens? ✅ **Only the CLS token** (the single global summary vector). We did **not** use patch tokens. This actually matches our failure hypothesis: the CLS token discards fine spatial detail, which precise length needs. **Trying patch tokens (which keep spatial detail) is a natural improvement we are now testing.**

Q: The poster implies "same parameters as the paper," but GitHub shows different learning rates. ✅ Fair point, now clarified on the poster. The truth: the **baseline reproduction** uses the paper's setup (Adam at 1e-3). The **encoder swaps deliberately use lower learning rates** (1e-4, and 1e-5/1e-6 for DINOv2 full fine-tune) because pretrained foundation models break if fine-tuned too aggressively. What is identical across all models is

the **data, split, input, head, loss, and metrics** — not the learning rate, which is set appropriately per encoder. That is correct practice, and the poster now says so explicitly.

PART G — Hyperparameters, and how we would tune them (beginner's guide)

What is a hyperparameter? A setting *you* choose before training, that the model does **not** learn by itself. (The things the model learns are called *parameters* or *weights*; the things you set are *hyperparameters*.) Think of baking: the recipe's oven temperature and baking time are hyperparameters; how the cake actually rises is what the model "learns."

Why tune them? The same model can give very different results depending on these settings. Tuning means searching for the combination that gives the lowest error on the **validation** set (never the test set — that stays for the final exam). We currently used sensible fixed values, not a search, which is one reason our small differences aren't final. ⚠️

The main hyperparameters in THIS project (what we used) ✅

Hyperparameter	What it does	Baseline	Encoder swaps
Learning rate	size of each learning step	1e-3	1e-4 (DINOv2 full: 1e-5 / 1e-6)
Batch size	fish seen before each update	32	16
Epochs	passes over the training data	200	100
Optimizer	the update rule	Adam	Adam
Loss	how error is measured	L1	L1
Head size	shape of the decision-maker	[1000,500,1]	[512,128,1]
Adaptation	how much of the eye trains	full	frozen / last-block / full
Image size	crop resolution	224	224
Augmentation	random color jitter on training	on	on

How to actually tune each model (plain steps)

The general recipe is the same for every model; only the sensible ranges differ:

1. **Pick one hyperparameter to vary** (learning rate is almost always the most important).
2. **Try a few values** (e.g. learning rate 3e-4, 1e-4, 3e-5).
3. **Train each, score on validation**, keep the best.
4. **Then vary the next one** (e.g. epochs, or head size), and repeat.
5. **Only at the very end**, run the single best setting on the test set.
6. To be rigorous, **repeat the winner with 2–3 random seeds** to be sure the win is real, not luck.

Per-model guidance (why the ranges differ): - **MobileNetV2 / ConvNeXt (supervised CNNs):** can take a **higher** learning rate (1e-3 to 1e-4) because we fully retrain them; worth trying more epochs and light weight-decay. 💡 - **CLIP (fine-tuned transformer):** use a **low** learning rate (1e-4 to 1e-5) so its pretrained knowledge isn't destroyed; the big lever is **how many blocks to unfreeze** (we tried last-block; trying the last 2–3 blocks is the natural next step). 💡 - **DINOv2 (self-supervised transformer):** most sensitive — it was *worst* at 1e-6 (too

tiny to learn) and unstable when fully fine-tuned. The high-value knobs here are **using patch tokens instead of the CLS token**, and **unfreezing the last few blocks at ~1e-4 to 1e-5**. 💡

Honest note for the meeting: we deliberately did **not** run a big hyperparameter search yet, because our first goal was a *fair, controlled* comparison (same settings across encoders). Per-model tuning would likely help the challengers more than the baseline, and is a clear next step. ✅ ⚠️

PART H — What we completed vs what's new/running

Completed ✅: dataset prep, masked square crops, official split, leakage audit (fish 113), baseline reproduction (0.633 vs 0.62), all 8 length experiments (4 encoders), server training, metric evaluation, ranking, error analysis by species/size/occlusion, qualitative disagreement figures, documentation and poster.

New / in progress (this round): - ⏳ Patch-token DINOv2 (tests the CLS-vs-patch question and our failure hypothesis). - ⏳ Species classification across encoders (tests if the trend holds on a second task). - ⏳ DINOv3 backbone (if weights available). - ⚠️ Mask segmentation — a **large** task (the paper used a heavy Mask2Former model); scoped carefully, likely a bigger follow-up rather than an overnight run.

Still open ⚠️: multi-seed repeats, confidence intervals / significance tests, EfficientNet, bigger foundation-model variants, real-time/deployment, cross-dataset testing.

PART I — What I can safely say vs must avoid

Safe to say (defensible): 1. The baseline was reproduced within 0.013 cm on non-occluded fish. ✅ 2. On this setup, no tested foundation model beat the baseline on average. 📊 3. CLIP transferred clearly better than DINOv2. 📊 4. Partial fine-tuning improved both CLIP and DINOv2. 📊 5. On individual hard (occluded) fish, CLIP is competitive with MobileNetV2 (wins ~40%). 📊

Must avoid: 1. "CNNs are always better than transformers." (Only a few small models, one dataset.) 2. "Foundation models can't do regression." (CLIP reached 0.958 cm.) 3. "MobileNetV2 is the best possible model." 4. "The ranking is statistically proven." (Single runs.) 5. Stating any *why* (e.g. DINOv2's flatfish failure) as fact — those are hypotheses.

PART J — Likely meeting questions (quick answers)

- **"Only length so far?"** → Yes; species classification is now running, segmentation is scoped as a larger task; we'll see if the trend holds.
- **"Show me where models differ."** → Point to the three figures in `results/qualitative/`: DINOv2 fails on small flatfish; CLIP wins on some occluded fish.
- **"When does CLIP beat MobileNetV2?"** → On ~39% of individual fish, more on occluded ones; it's competitive on hard cases even though MobileNetV2 wins on average.
- **"DINOv3? Patch tokens?"** → No DINOv3 yet; CLS token only so far; patch-token DINOv2 is now running because it may fix the spatial-detail weakness.
- **"Same parameters as the paper?"** → Baseline yes; foundation models use lower learning rates by design; data/split/head/metrics identical. Poster now states this clearly.

- **"Did you tune hyperparameters?"** → Not a full search yet, on purpose, to keep the comparison fair; per-model tuning is a defined next step (see Part G).

This guide reflects the state at the start of the new experiment round. A results-updated version will follow once the patch-token, species-classification, and DINOv3 runs finish.